Anjuman-I-Islam's

M. H. SABOO SIDDIK COLLEGE OF ENGINEERING

8, Saboo Siddik Polytechnic Road, Byculla, Mumbai, Maharashtra 400008

DEPARTMENT OF INFORMATION TECHNOLOGY

ROSP

Final Year Project (Semester VII)

Title of the Project

"OMARCHY STUDIO"

A Unified, Undo-Safe Desktop Configuration Studio for Omarchy Linux

Guide

"Prof. Fardeen "

Submitted by

Naved Rar (231433)

REV - 2019 'C' Scheme

University of Mumbai

Academic Year (2026 -27)

CERTIFICATE

This is to certify that the project entitled **“Omarchy Studio”** is a bonafide work of **“Aariz Sheikh”** (231453), **“Yusuf Mohd Aslam”** (231460), **“Samia Loladia”** (231463), **“Naved Rar”** (231433) submitted to the **University of Mumbai** in partial fulfillment of the requirement for the **ROSP – Final Year Project** of the **7TH Semester** (Final Year, Bachelor of Engineering) in the **Department of Information Technology**.

(PROF. FARDEEN)

SUPERVISOR/GUIDE

(DR. ZAINAB MIRZA) **(PROF. FARDEEN)**

HEAD OF DEPARTMENT **ROSP CO-ORDINATOR**

Place : Mumbai

Date :

Project Report Approval

This project report entitled **“Omarchy Studio”** by **Aariz Sheikh, Yusuf Mohd Aslam, Samia Loladia & Naved Rar** is approved for the **ROSP Final Year Project** of the 7th Semester in the Department of Information Technology.

Examiners:

---------- (Internal Examiner)

---------- (External Examiner)

Date:

Place: Mumbai

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

NAVED RAR 231433

Project team:

Name	Roll No.	Signature
AARIZ SHEIKH	231453	
YUSUF MOHD ASLAM	231460	
SAMIA LOLADIA	231463	
NAVED RAR	231433	

Date:

ACKNOWLEDGEMENT

We, the project team — **Aariz Sheikh, Yusuf Mohd Aslam, Samia Loladia and Naved Rar** — would like to express our heartfelt gratitude to everyone who supported and guided us throughout the development of our **ROSP Final Year Project, “Omarchy Studio”**.

We are especially thankful to our project guide, **Prof. Fardeen Sir**, for his continuous support, expert guidance and valuable feedback at every stage of the project. His mentorship played a crucial role in shaping our ideas, in insisting on careful engineering and testing, and in helping us overcome the challenges we met while building a tool that edits a live desktop safely.

We are sincerely grateful to **Dr. Zainab Mirza Ma'am**, Head of the Department of Information Technology, for her encouragement and for providing the facilities and environment that made this work possible.

We also extend our sincere thanks to the faculty and staff of the **Information Technology Department at M.H. Saboo Siddik College of Engineering** for providing us with the necessary resources and a productive learning environment, and to the open-source communities behind Omarchy, Hyprland, Rust and Ratatui, whose documentation and code we learned from.

Our sincere appreciation goes to our friends and classmates for their inputs, testing and motivation. Last but not least, we are grateful to our families for their patience, encouragement and constant support during this journey.

This project was a great learning experience for all of us, and we are truly thankful to everyone who contributed directly or indirectly to its success.

Student Information

Name	Roll No.
AARIZ SHEIKH	231453
YUSUF MOHD ASLAM	231460
SAMIA LOLADIA	231463
NAVED RAR	231433

Role	Name
Project Guide	Prof. Fardeen Sir
ROSP Co-ordinator	Prof. Fardeen Sir
Head of Department	Dr. Zainab Mirza Ma'am

Abstract

Omarchy Studio is a keyboard-driven desktop configuration studio for **Omarchy**, an opinionated Arch Linux distribution built on the **Hyprland** Wayland compositor. Omarchy lets a user pick a theme from its menu, but every other customisation — keybindings, window gaps and blur, animations, the status bar, notifications, the lock screen, idle timers and displays — still means hand-editing files in five different configuration dialects spread across four directory trees, with no discoverability and no undo. One typo in a compositor file can leave the user with a session that will not reload.

Studio replaces that workflow with a single terminal user interface (TUI) and a matching command-line interface (CLI). It covers more than twenty modules: themes with live preview and a 114-theme community browser, a wallpaper browser with wallhaven.cc search, a wizard that extracts a complete colour theme from any image using median-cut quantisation, keybinds with live chord capture and conflict detection, 56 Hyprland look-and-feel settings with instant preview, animation presets, bar, notification, OSD and idle editors, monitor arrangement, power and battery thresholds, safe application removal with a dependency cascade preview, and a health-checking Doctor.

The system is engineered around trust. Every change passes through one **apply pipeline** — plan, drift check, pre-snapshot, hash-guarded atomic write, reload, verification and post-snapshot — and is **rolled back automatically** if the compositor reports configuration errors or a restarted component dies. Configuration files are edited with **lossless, comment-preserving parsers**, so the user's own edits survive byte-for-byte, and Omarchy's vendored files are never touched, so system updates cannot clobber the user's work. Every change is committed to a local **git-backed snapshot store** that can be browsed as a timeline with coloured diffs and restored to any point.

Studio is written in **Rust** as a two-crate workspace (an engine library and a TUI/CLI binary built on Ratatui and clap) of roughly 43,700 lines. It is verified by 477 automated tests, terminal end-to-end journeys run in CI, and has been released publicly as open-source software (MIT licence), supporting both Omarchy 3.8 and the Lua-configured Omarchy 4 “Quattro”.

Table of Contents

Sr. No.	Section	Page no.
1	Introduction	1
2	Literature Survey	3
3	Proposed System Architecture	5
4	Technology Stack	12
5	Diagrams	14
6	Results and Screenshots	19
7	Comparative Analysis	29
8	Conclusion	31
9	Future Scope	32
10	References	33

1. Introduction

Linux desktop customisation — popularly called “ricing” — has become one of the main reasons people adopt tiling Wayland compositors such as Hyprland. **Omarchy**, an opinionated Arch Linux setup published by David Heinemeier Hansson, packages Hyprland with a curated set of applications and beautiful themes, and has attracted a large community of new Linux users. Omarchy's menu lets a user switch the theme in one keystroke, but everything beyond theme selection drops the user into a text editor.

A typical customisation session therefore requires knowledge of **five configuration dialects** — Hyprland's hyprlang (or Lua on Omarchy 4) for keybinds and window behaviour, JSONC and GTK-CSS for the Waybar status bar, INI for mako notifications and TOML for colour palettes — spread across four directory trees. Some of those files are owned by Omarchy and are silently replaced by `omarchy-update`; others are the user's to edit. There is no list of the settings that exist, no indication of which file a keybinding came from, no preview of a change before it is saved, and no undo. A single typo in `hyprland.conf` can break the compositor reload.

Omarchy Studio addresses these problems with a single keyboard-driven terminal application and a matching CLI that together form a “control centre” for the whole desktop. Every editable knob is presented as a plain-language field with a live preview; the generated configuration line is shown alongside so the user learns the underlying syntax passively. Every change is written through one verified apply pipeline that snapshots the files to a local git repository first and rolls the change back automatically if the desktop rejects it. Studio never writes to Omarchy's own files, so system updates and Studio's changes cannot fight each other.

1.1 Problem Statement

- **The configuration cliff** — beyond theme selection, every change requires hand-editing files in dialects most users do not know.
- **No discoverability** — which bar modules exist, which animation curves are possible, which of six sourced binding files owns a shortcut: today the answer is “read the source”.

- **No safety net** — a typo breaks the compositor reload, edits to Omarchy-managed files are clobbered on update, and nothing can be undone.
- **Fragmented tooling** — colours are solved by half a dozen tools, monitors by two, while keybinds, animations, bar layout and notification behaviour have no tool at all.

1.2 Objectives

1. Provide zero-jargon editing of every user-facing Omarchy surface through forms, pickers and live previews.
2. Make every change reversible: snapshot before and after, one-key undo, and a browsable history.
3. Verify each change against the running desktop and roll back automatically on failure.
4. Preserve the user's hand-edits byte-for-byte using lossless, comment-preserving parsers.
5. Integrate natively with Omarchy (menu entry, lifecycle hooks) and survive `omarchy-update`.
6. Offer a scriptable CLI twin for every screen, driven by the same engine.

1.3 Scope

The project targets Omarchy 3.8 (hyprlang configuration) and Omarchy 4.0 “Quattro” (Lua configuration, Quickshell-based shell) on Hyprland 0.55–0.56. It is a configuration tool for the user's own desktop; it is not a general dotfile manager, an image editor, a package manager or a daemon — Studio runs, applies and exits, and lifecycle hooks handle the rest.

2. Literature Survey

Configuration errors are one of the dominant causes of failures in real systems. In [1], Xu and Zhou survey systems approaches to tackling configuration errors and organise the field into prevention, testing, diagnosis and troubleshooting. They argue that configuration should be treated as a first-class input that is validated before use and that recovery mechanisms such as checkpointing and rollback are essential because many misconfigurations can only be detected at run time. Studio follows exactly this model: every write is validated by its dialect parser before it reaches disk, verified against the running compositor after reload, and rolled back from a snapshot on failure.

Yin et al. [2] studied hundreds of real configuration errors from commercial and open-source systems and found that a large fraction were simple parameter mistakes made by users, and that systems frequently gave poor or misleading feedback when a setting was wrong. Their recommendation — pinpoint the offending parameter and explain it in the user's terms — motivated Studio's error model, in which every failure is rendered as "what happened → why → what to do next" and mapped back to the exact field on screen.

In [3], Xu et al. show that system software is frequently over-configured: most users touch only a small subset of the available knobs, and the sheer number of options is itself a source of errors. Studio responds by curating settings into grouped, documented schemas (56 Hyprland settings in eight categories, twelve mako behaviour settings) with sensible bounds, each carrying a one-line human description, instead of exposing every raw key.

Shneiderman's guidelines for interface design [4] include "permit easy reversal of actions" and "offer informative feedback"; direct manipulation with immediate, visible results lowers the anxiety of exploration. Studio applies these principles through live preview (every Look & Feel adjustment is pushed to the compositor with `hyprctl keyword` before it is saved), a ten-second confirm-or-revert countdown for risky changes, and one-key undo backed by a git history.

For theme generation, Heckbert's median-cut algorithm [5] recursively splits the colour space along its longest axis to find a small representative palette of an image. Studio's palette extraction ports the median-cut approach (with Normal, Muted and Material modes inspired by the MIT-licensed Aether project [15]) and assigns the extracted colours to Omarchy's 21-slot `colors.toml`. Contrast between foreground, accent and background is checked against the WCAG 2.1 contrast-ratio formula [6] so that generated themes remain readable.

2.1 Existing Tools in the Omarchy Ecosystem

The Omarchy community has produced several tools, each solving one slice of the problem. Table 1 summarises them; a detailed comparison with Studio appears in Chapter 7.

Table 1: Existing customisation tools for Omarchy

Tool	Approach	Limitation
Omarchy menu [7]	Bash + Walker menus; theme pick/install, then “open this file in nvim”	No structured editing, preview or undo beyond themes
Aether [15]	Go + Wails GUI; wallpaper → palette extraction, wallhaven browser, presets	Colours only — no keybinds, animations, bar or notifications
Omarchist [17]	Rust + GPUI visual theme designer	Early stage; no keybinds, animations or notifications
hyprmon [18]	Go TUI for monitor layout	Monitors only
A La Carchy [14]	Single 10.8k-line bash script: debloat + many toggles	Edits some vendored files; tar.gz backup instead of verified rollback; no tests

None of these tools combines the visual half of ricing (colours, wallpapers) with the behavioural half (keybinds, animations, bar, notifications, displays), and none offers verified apply with automatic rollback. That gap defines Omarchy Studio.

3. Proposed System Architecture

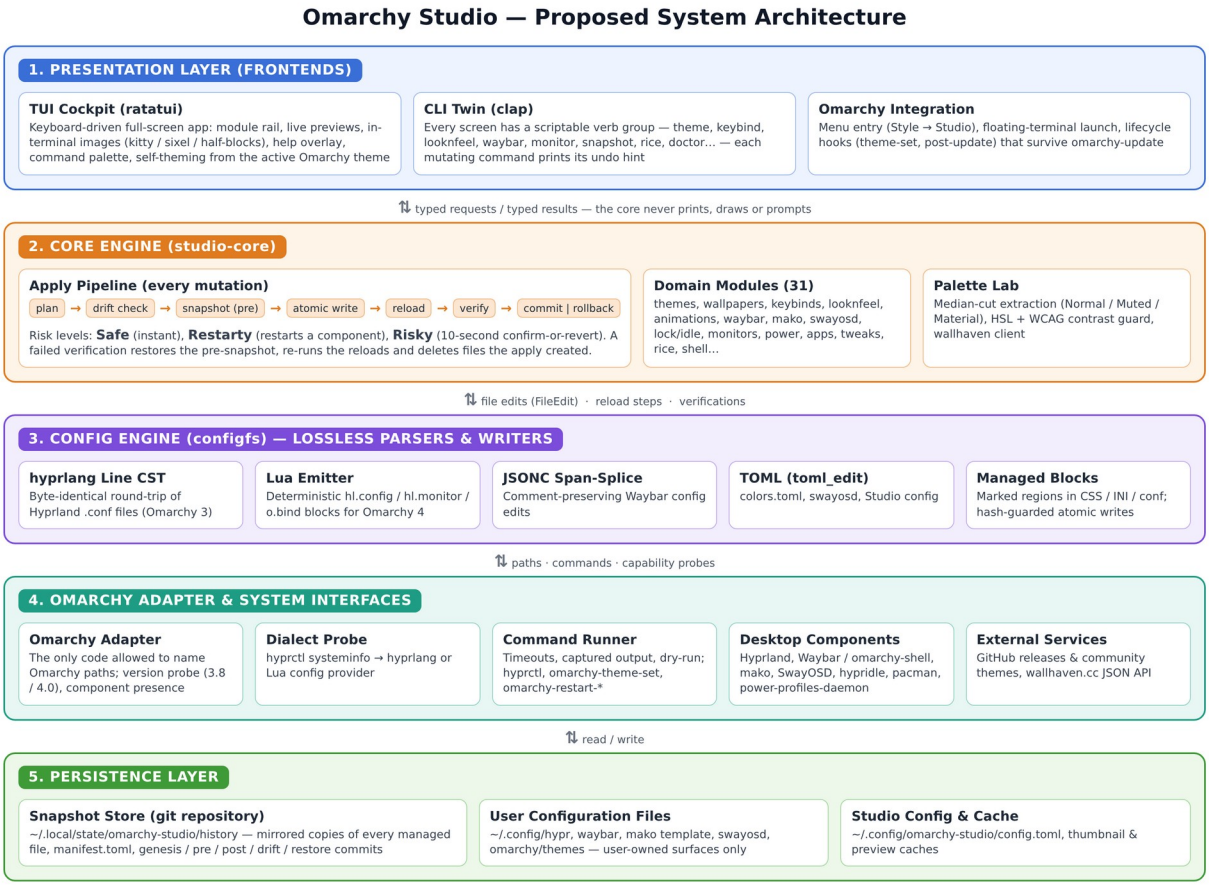


Figure 1: Layered architecture of Omarchy Studio

It Includes

1. Presentation Layer (TUI Cockpit, CLI Twin, Omarchy integration)
2. Core Engine (studio-core: apply pipeline, domain modules, palette lab)
3. Config Engine (lossless parsers and writers)
4. Omarchy Adapter & System Interfaces
5. Persistence Layer (snapshot store and configuration files)

Presentation Layer (Frontends) - Detailed Analysis

The Presentation Layer is the only part of Studio that talks to the user. It contains two frontends over the same engine, and the rule that keeps them honest is that **studio-core**

never prints, never draws and never prompts — it returns typed results, and the frontends render them. This is also what keeps a future graphical frontend cheap to build.

TUI Cockpit

The TUI is a full-screen Ratatui application with a module rail on the left, the active screen on the right and a key bar at the bottom. Studio themes itself from the active Omarchy theme, so its panels and highlights re-tint with every theme switch. Images (theme wallpapers, wallhaven thumbnails) are drawn inside the terminal with the kitty or sixel graphics protocols, falling back to half-block characters.

User Workflow:

1. **Launch**: from the Omarchy menu (Style → Studio), a keybinding or a terminal; Studio opens as a floating window like Omarchy's own TUIs.
2. **Onboarding**: on first run a welcome card offers three optional steps — track every change, wire into Omarchy (menu entry and update-survival hooks) and theme terminal tools.
3. **Navigate**: `Tab`/`Shift+Tab`, `1`–`9`, arrow keys or a mouse click moves between modules; `/` opens a fuzzy command palette; `?` shows context help generated from each screen's key map.
4. **Edit with live preview**: values are adjusted with `h`/`l`; compositor settings are pushed live, themes preview their wallpaper and palette, keybinds are captured by pressing the chord itself.
5. **Save**: `s` sends the pending edits through the apply pipeline; the result is confirmed with “Applied — u to undo”.
6. **Undo / restore**: `u` undoes the last change; the Snapshots screen shows every change with a coloured diff and restores any point in time.

CLI Twin

Everything the TUI does is scriptable through a clap-based command tree with 30 verb groups (`theme`, `wallpaper`, `keybind`, `looknfeel`, `animations`, `waybar`, `monitor`, `snapshot`, `rice`, `doctor`, …). Each mutating command prints its undo hint, e.g. `looknfeel general.gaps_in=8 · undo with omarchy-studio snapshot`

`undo`. Because both frontends call the same core functions, a change made in the CLI is undoable from the TUI and vice versa.

Core Engine

The Core Engine (`studio-core`) contains all domain logic — 31 module files covering themes, wallpapers, keybinds, look & feel, animations, Waybar, mako, SwayOSD, lock & idle, monitors, power, battery, apps, tweaks, rice bundles, theme sync, the Omarchy 4 shell and the self-updater — and the machinery they all share.

Settings schema. Every editable field is a `Setting` with an identifier, a target (file, dialect and key path), a widget (stepper, toggle, colour, select, chord or text), a one-line human explanation, an optional live-apply action such as `HyprctlKeyword("decoration:blur:size")`, dependency gates and version gates. The TUI renders forms from the schema and the CLI resolves `get`/`set` paths against it, so adding a setting means adding data, not UI code.

The apply pipeline. Every mutation is described by an `ApplyPlan` containing the file edits, the ordered reload steps (`hyprctl reload`, `omarchy-theme-refresh`, restart a component, reload mako), the verifications to run afterwards and a risk level. The pipeline executes **plan → drift check → snapshot (pre) → atomic write → reload → verify → commit or rollback**. Verifications include `hyprctl configerrors` being empty, a process (such as Waybar) still being alive after a grace period, and arbitrary commands exiting successfully. Risky changes arm a ten-second confirm-or-revert countdown, the same pattern operating systems use for display settings.

Error model. Errors form a typed taxonomy — missing dependency, parse failure with line and hint, verification failure (with whether rollback succeeded), Omarchy version mismatch, I/O and external command failures. The frontend contract is that every error renders as *what happened → why → what to do next*; a missing dependency is never an error at all but a visible, disabled feature carrying the exact install command.

Concurrency. The core is single-threaded; worker threads are used only for wallhaven HTTP requests and thumbnail decoding, image previews, palette extraction and command execution with timeouts, communicating with the UI over `std::sync::mpsc`

channels. No async runtime is needed because every system command Studio runs is a fast local process.

Config Engine (Lossless Parsers)

The Config Engine (`configfs`) is the trust core of the system. Its guiding principle is to parse fully only what Studio must understand and, for everything else, to own a clearly marked region and treat the rest of the file as opaque text. Table 2 lists the strategy per dialect.

Table 2: Configuration dialects and editing strategies

Dialect	Files	Strategy
hyprlang	~/.config/hypr/*.conf (Omarchy 3)	Line-level CST; byte-identical round-trip
Lua	~/.config/hypr/*.lua (Omarchy 4)	Emit-only managed block of hl.* calls; values read back via hyprctl
TOML	colors.toml, swayosd config, Studio config	Lossless document edit with toml_edit
JSONC	waybar/config.jsonc, omarchy-menu.jsonc	Span-splice editor preserving comments and order
INI	themed/mako.ini.tpl	Studio-owned file, simple line model
CSS	waybar/style.css, swayosd/style.css	Managed blocks only — never a full CSS parse

hyprlang line CST. A Hyprland file is parsed into a lossless list of lines — blank, comment, assignment (with category path and trailing comment), category open/close, `source`, variable or unknown — where unknown lines are never dropped or touched. The invariant `parse(s).to_string() == s` is tested against real configuration files.

Lua emission for Omarchy 4. Omarchy 4 moved Hyprland's configuration to Lua. Studio emits a deterministic managed block of `hl.config{}`, `hl.monitor{}`, `hl.curve`/`hl.animation` and `o.bind` calls at the end of the user's file (later calls deep-merge over earlier ones), never parses arbitrary Lua, and reads effective values back

through `hyprctl -j --batch getoption`. Every emitted block is syntax-checked by the real `luac` in the test suite.

Managed blocks and atomic writes. Shared files receive a marked block such as `/* >>> omarchy-studio:<section> … >>> */` that is fully regenerated on each apply, while everything outside it is untouchable. Writes are hash-guarded (the write is refused if the file changed since it was parsed), go to a temporary file, are flushed with `fsync` and then atomically `renamed` into place. Symlinked theme files are never written through.

Safety & Verification Layer

Safety in Studio is not a single component but a set of guarantees enforced at the boundaries of every layer. Together they ensure that an experiment can never leave the user with a broken desktop.

- **Verified apply with automatic rollback** — after every reload the pipeline checks `hyprctl configerrors`; on failure it restores the pre-snapshot files, deletes files the apply created and re-runs the reload steps.
- **Waybar crash watchdog** — a bar configuration is applied, Waybar is restarted and, if the process dies within the grace period, the previous configuration is restored automatically.
- **Drift detection** — if a managed file was hand-edited since Studio's last snapshot, a `drift` snapshot is recorded first, so the user's own edit is preserved in history too.
- **Never fight Omarchy** — Studio writes only user-owned files and sanctioned extension surfaces (`themed/*.tpl`, hooks, menu extensions); a CI check fails the build if any code outside the Omarchy adapter names Omarchy's vendored path.
- **Version awareness** — the adapter probes the Omarchy and Hyprland versions and the configuration dialect; untested versions produce a warning but never a refusal.
- **Privileged writes are previewed** — the AC/battery power rule and persistent battery thresholds show the exact root write for approval before anything runs; app removal shows the full `pacman -Rs` cascade and never bypasses pacman's refusal to break dependencies.

Persistence Layer — Snapshot Store

The snapshot store is a plain git repository at `~/.local/state/omarchy-studio/history/`. It mirrors every managed file under `files/` at its absolute path and keeps a `manifest.toml` recording which module claimed each file. Each apply produces a `pre:` commit before the write and a `post:` commit after successful verification, with the change summary as the subject (for example `looknfeel general.gaps_in=8`) and optional `key: value` trailers. Hand-edits are captured as `drift:` commits, the first state as `genesis:` and every undo as a `restore:` commit — history is append-only, so an undo can itself be undone.

Using the system `git` binary rather than a library keeps the executable small, and git is always present on Omarchy. The same store powers three user-facing features: one-key undo, the Snapshots timeline (with per-commit coloured diffs and whole-tree or per-file restore), and **rice bundles** — `rice export` archives every file Studio has written, grouped by module, and `rice import` replays each file through the full apply pipeline on another machine instead of untarring over a live desktop.

Theme & Palette Pipeline

The theme-from-wallpaper wizard turns any image into a complete, standard Omarchy theme. The image is sampled and quantised with median cut into a representative palette; the selected mode (Normal, Muted or Material) and light/dark bias shape the result; an accent is chosen and checked for WCAG contrast against the background; and the colours are written to all 21 keys of `colors.toml` together with the wallpaper and a generated `preview.png`. The wizard is reachable from any wallpaper (`t`), from wallhaven search results and from the CLI (`theme new --from-image`, `theme new --from-current-wallpaper`). The community browser complements it with 114 published themes whose wallpaper and palette are previewed *before* anything is installed.

Omarchy 4 “Quattro” Adapter

Omarchy 4 changed the platform underneath Studio: Hyprland configuration moved from hyprlang to Lua, the menu moved from a bash script to a Quickshell plugin reading a

JSONC file, and Waybar, mako, SwayOSD and hypridle were replaced by the Quickshell-based `omarchy-shell`. Studio handles both generations from one binary: the dialect is probed from the running compositor (`hyprctl systeminfo` → `configProvider`), every Hyprland writer renders hyprlang or Lua accordingly, retired components are detected and their screens explain why they are unavailable, and a new shell module edits the Omarchy 4 bar layout, plugins, idle timers and appearance by wrapping Omarchy's own `omarchy bar` and `omarchy plugin` commands.

4. Technology Stack

The proposed system is a native, dependency-light application optimised for safety, speed and distribution as a single binary. The technology stack is divided into four layers.

Language & Frontends

Studio is written in **Rust (edition 2021, toolchain 1.96)**, chosen for memory safety, strong typing of the configuration model and a single statically-linked-feeling binary that users download once. The TUI is built on **Ratatui 0.29** with its re-exported **crossterm** backend, and terminal image previews use **ratatui-image** (kitty, sixel and half-block protocols). The CLI uses **clap 4** for the command tree, help text and did-you-mean suggestions.

Configuration & Data

Lossless TOML editing uses **toml_edit**; JSON from `hyprctl -j` and the wallhaven API is decoded with **serde / serde_json**. The hyprlang CST, the JSONC span-splice editor, the INI model and the Lua emitter are hand-written so that every byte of the user's files can be preserved. The **image** crate decodes PNG, JPEG, GIF and WebP for palette extraction and previews, and **ureq** (with rustls, no OpenSSL) makes blocking HTTP requests on worker threads.

System Integration

Studio drives the desktop through its own tools: **hyprctl** (keywords, reload, configerrors, binds, monitors), Omarchy's **omarchy-theme-set / omarchy-theme-refresh / omarchy-restart-*** primitives, **makoctl**, **swayosd-client**, **pacman** and **powerprofilesctl**. Snapshots are stored with the system **git** binary.

Quality, CI & Distribution

GitHub Actions runs rustfmt, clippy with warnings denied, the full test suite, an adapter-boundary check, a crates.io packaging check and tmux-driven end-to-end journeys on every push. Releases are built with link-time optimisation and distributed as a binary through a checksummed `curl | sh` installer, an **AUR** package and a built-in self-updater that defers to pacman for packaged installs.

Table 3: Technology stack summary

Layer	Technology	Purpose
Language	Rust 2021 (toolchain 1.96)	Memory-safe, typed engine and single binary
TUI	Ratatui 0.29 + crossterm	Full-screen keyboard-driven interface
Images	ratatui-image 8, image 0.25	In-terminal previews, palette extraction
CLI	clap 4	Command tree, help, suggestions
Config	toml_edit 0.22, serde_json, hand-written CST	Lossless, comment-preserving edits
Network	ureq 2 (rustls)	wallhaven API, GitHub releases & themes
History	git	Snapshot store, diffs, restore
Desktop	hyprctl, omarchy-* scripts, makoctl, pacman	Live preview, reload, verification
CI / Release	GitHub Actions, tmux, AUR	Lint, test, e2e, packaging

5. Diagrams

Flow Diagram

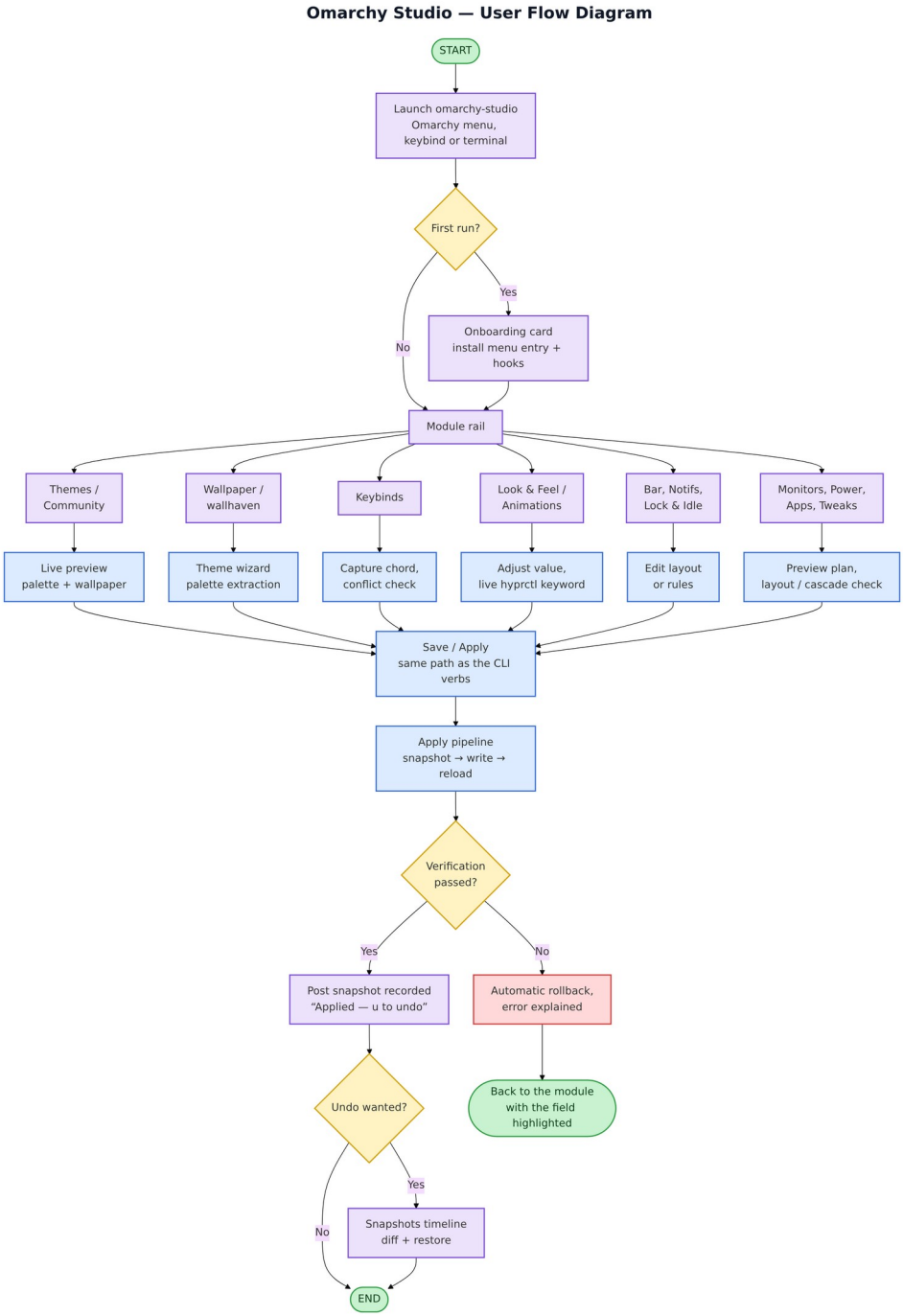


Figure 2: User flow through Omarchy Studio

Use Case Diagram

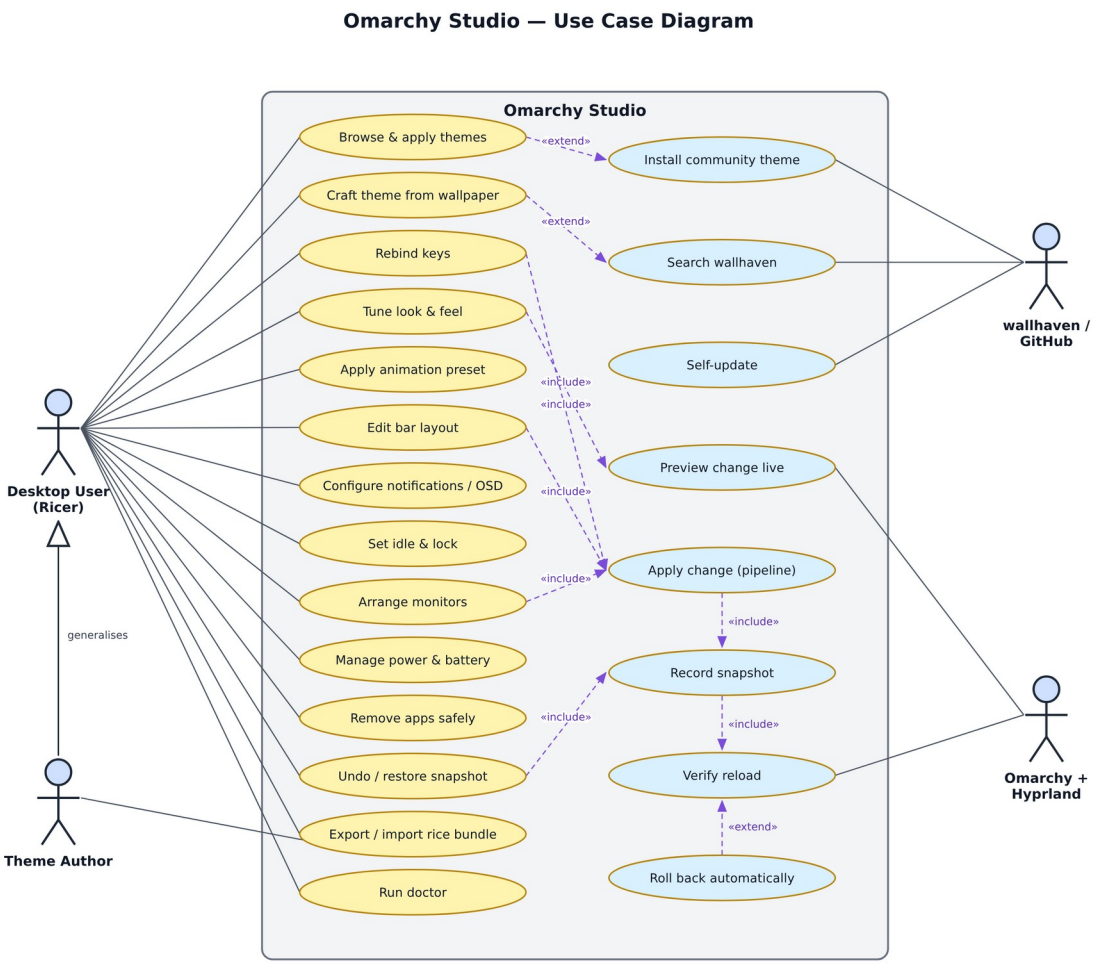


Figure 3: Use case diagram

Data Modelling Diagram

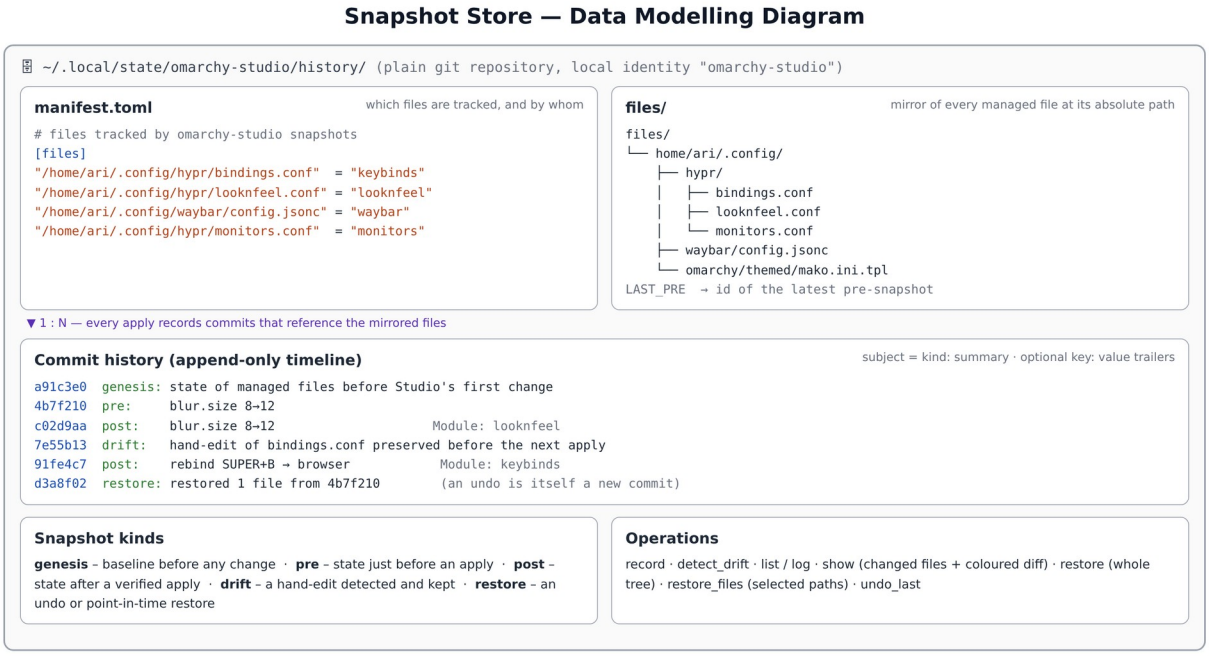


Figure 4: Data model of the git-backed snapshot store

Data Flow Diagram

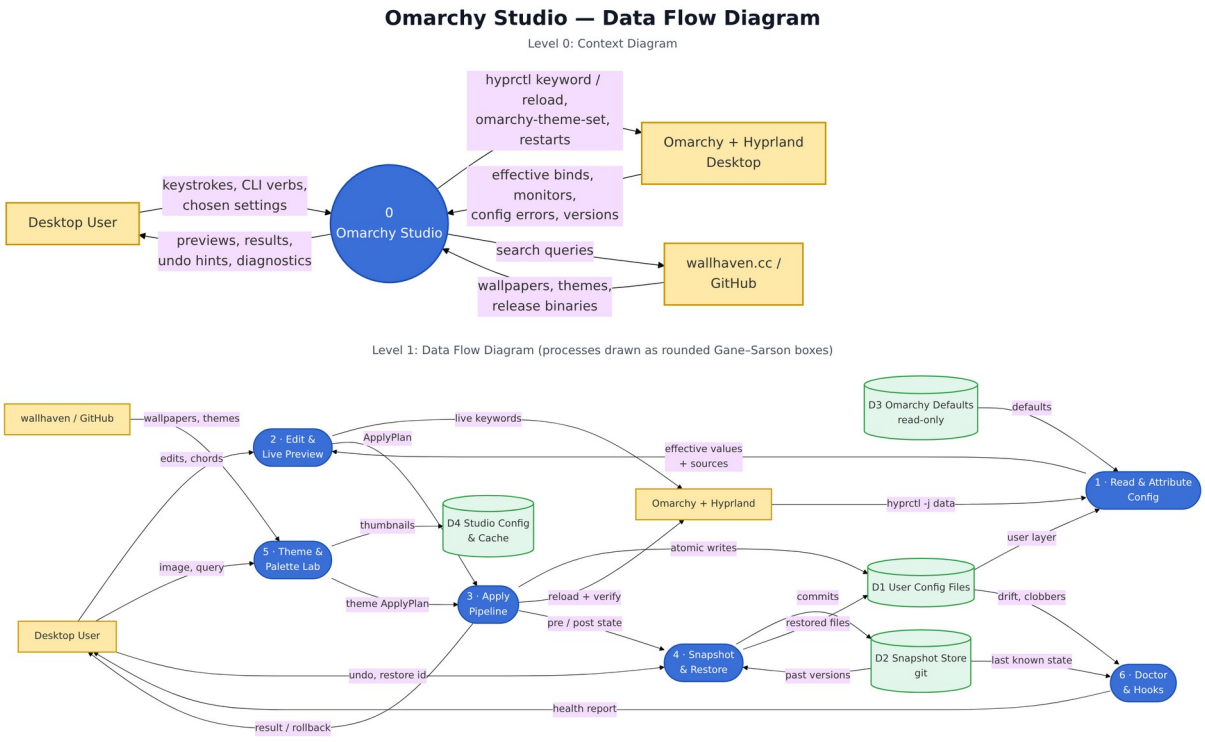


Figure 5: Data flow diagram — Level 0 and Level 1

Activity Diagram — Apply Pipeline

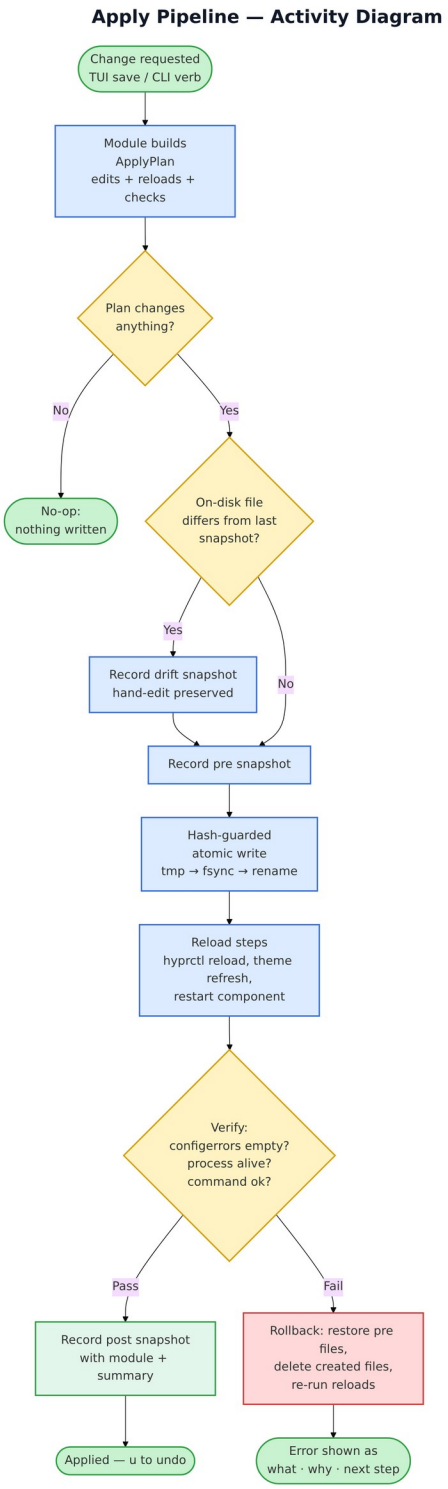


Figure 6: Activity diagram of the apply pipeline

6. Results and Screenshots

6.1 Test Results

Because Studio edits a live desktop, test effort was concentrated on the parts users must be able to trust. The full workspace suite was run for this report with `cargo test --workspace`:

Test suite	Tests	Result
studio-core unit tests (parsers, pipeline, snapshots, modules)	388	387 passed, 1 ignored
omarchy-studio unit tests (TUI screens, CLI)	79	79 passed
Golden round-trip tests (real config files)	10	10 passed
tmux end-to-end journeys (tools/e2e.sh)	—	All checks passed
Total automated tests	477	0 failures

The end-to-end journey launches the real release binary in a tmux pseudo-terminal against a fixture Omarchy installation and asserts on the captured screen: the first-run on-ramp, rail navigation, the panel following the selection, the help and search overlays and a clean exit. A separate rollback journey injects a fake `hyprctl configerrors` failure and proves that a rejected change — including a rejected *first-ever* write — leaves the user's files exactly as they were.

6.2 Codebase and Build Metrics

Metric	Value
Rust source (both crates, incl. tests)	≈ 43,700 lines
studio-core engine / omarchy-studio binary	≈ 25,500 / ≈ 17,900 lines
Domain module files / TUI screen files	31 / 23
CLI verb groups	30
Release binary size (LTO, debuginfo stripped)	≈ 7.3 MB (public release v0.9.3)

6.3 Feature Coverage

- **56** Hyprland look-and-feel settings in 8 categories, each with live preview; **6** look-and-feel presets and **6** animation presets.
- **41** Hyprland dispatchers given plain-language names in the keybind editor, with source attribution across every sourced binding file.
- **114** community themes browsable with a pre-install wallpaper and palette preview.
- **30** curated packages and web apps in the safe-removal catalogue with a dependency cascade preview.
- Verified on **Omarchy 3.8 and 4.0** with **Hyprland 0.55 and 0.56**; both hyprlang and Lua configuration dialects are written from one binary.

6.4 Screenshots

1. Themes Screen



The home screen: every installed theme with its palette strip, light-mode markers and a live preview of the selected theme's wallpaper and a mock terminal drawn in its colours.

2. Wallpaper Browser



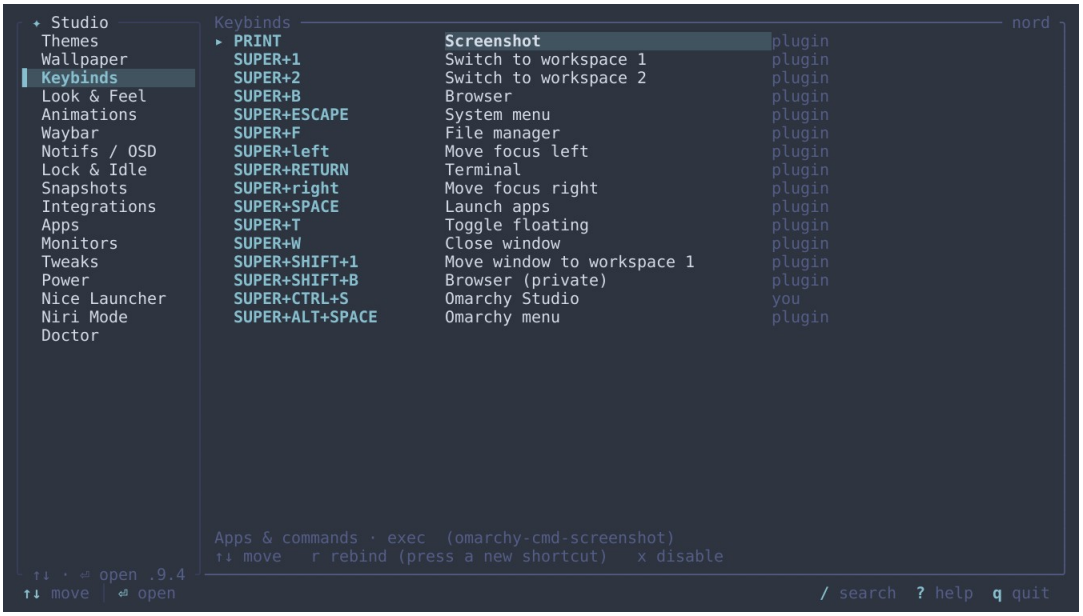
Wallpapers from all four sources Omarchy merges, with an in-terminal image preview; `w` opens wallhaven search and `t` crafts a theme from the selected image.

3. Theme-from-Wallpaper Wizard



Median-cut extraction turns the image into a full 21-colour palette; modes and light/dark bias can be switched with live re-extraction before the theme is saved.

4. Keybinds Screen



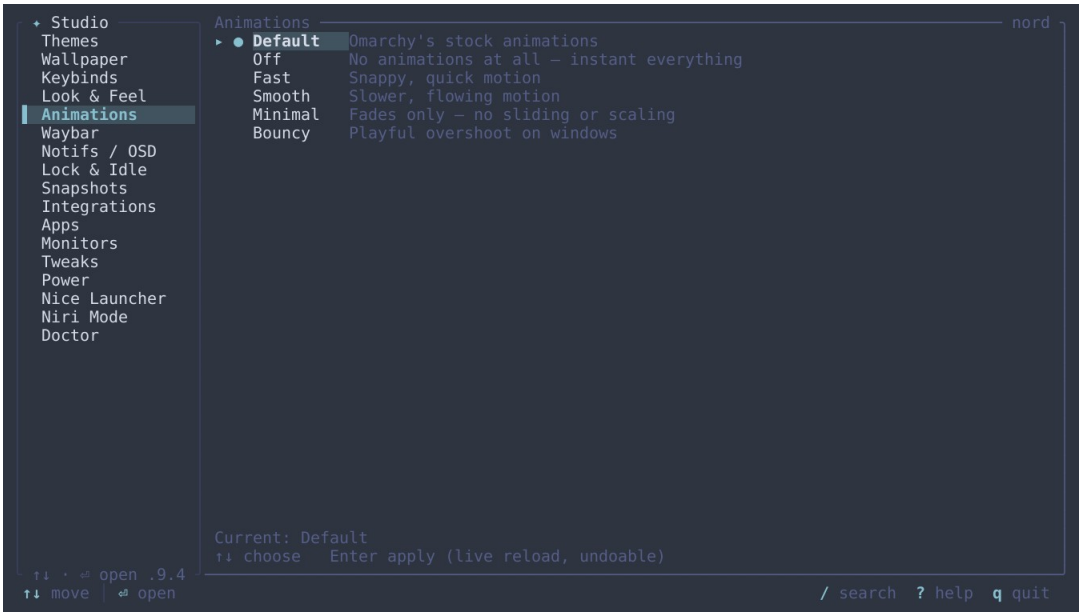
The effective keymap read from the compositor, with human-readable actions and the file each binding comes from; pressing `/` captures a new chord and conflicts are reported before saving.

5. Look & Feel Screen



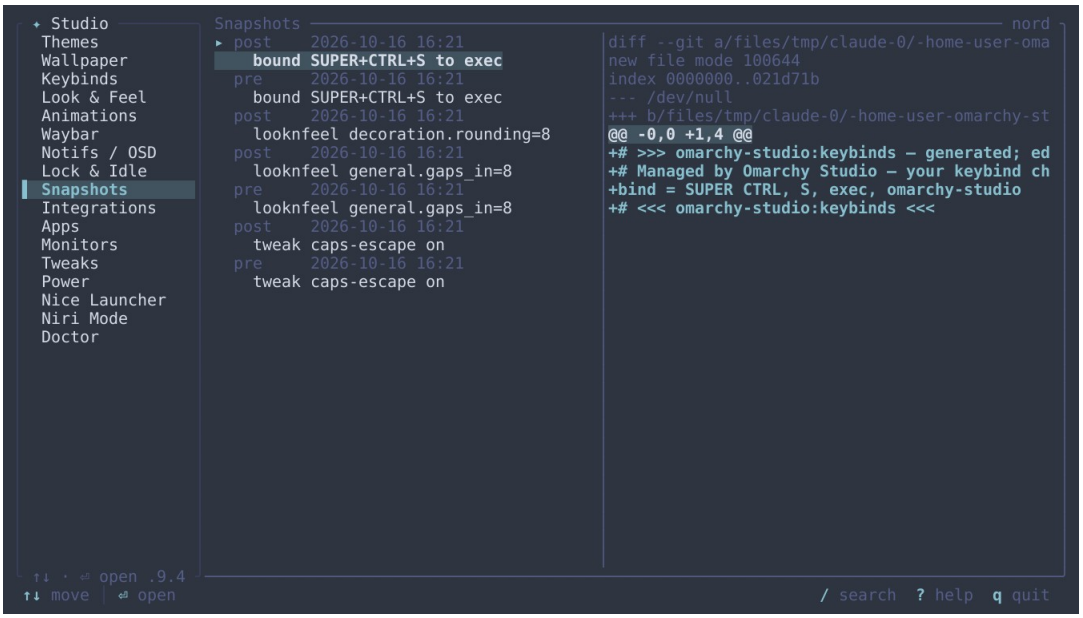
Hyprland settings grouped into tabs (Windows, Layout, Behavior, Corners & opacity, Blur, Shadow, Input, Touchpad); every adjustment is previewed live and `s` saves through the pipeline.

6. Animations Screen



Six hand-tuned animation feels from instant to bouncy, applied as a managed block with a live reload.

7. Snapshots Timeline



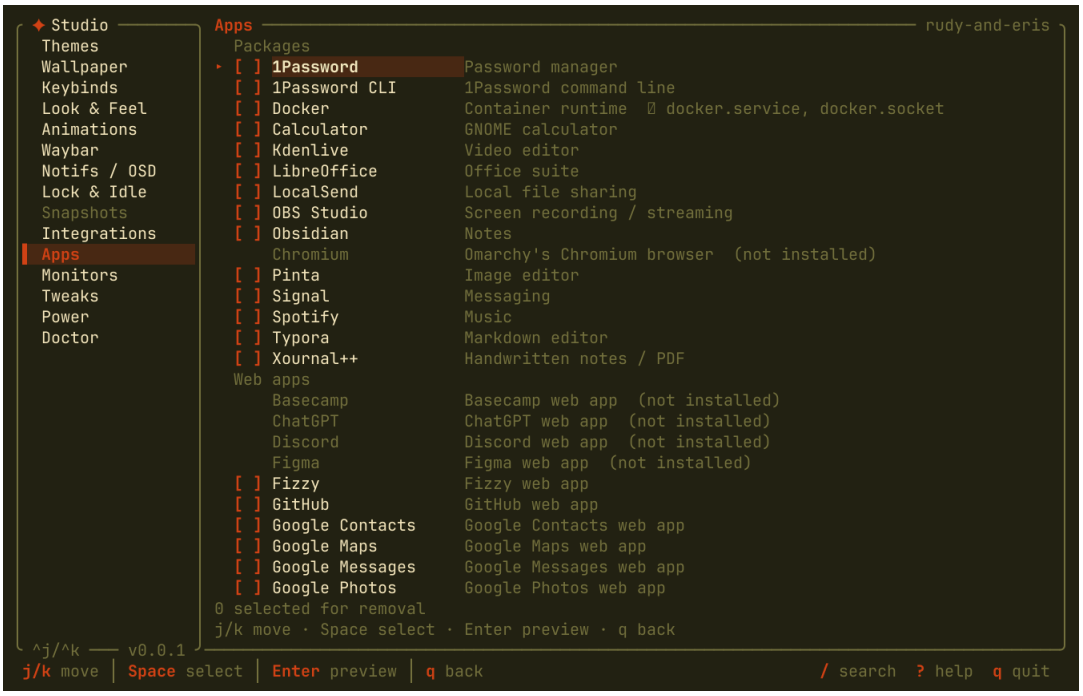
Every pre/post snapshot Studio has recorded, with the coloured diff of the selected change; any point can be restored, and the restore is itself recorded.

8. Monitors Screen



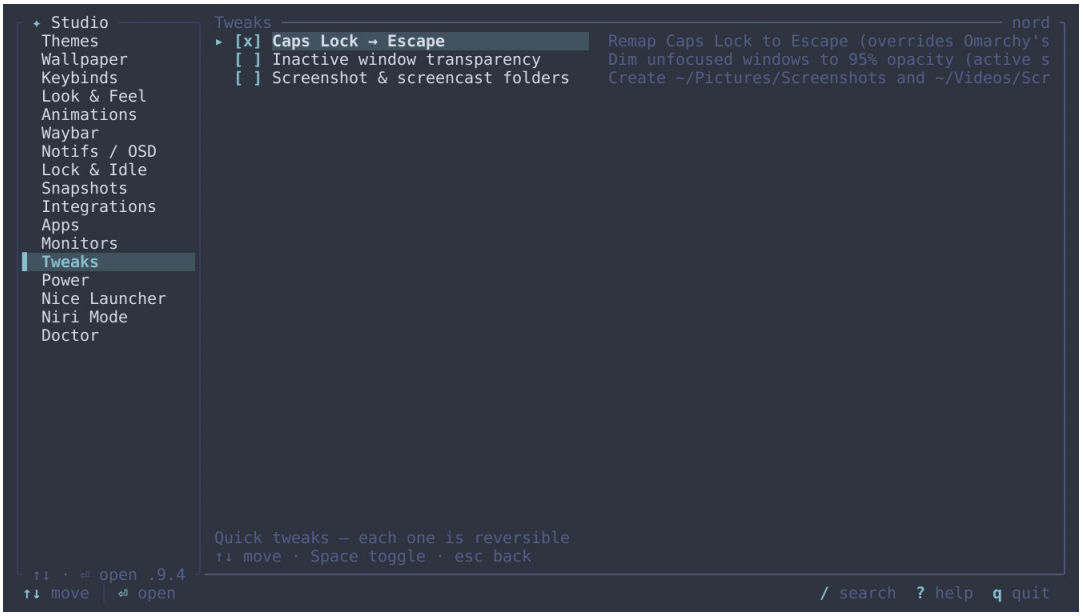
A proportional map of the connected displays with resolution, refresh rate, scale and effective size; placement mode arranges one screen against another and warns about overlaps.

9. Apps & Services



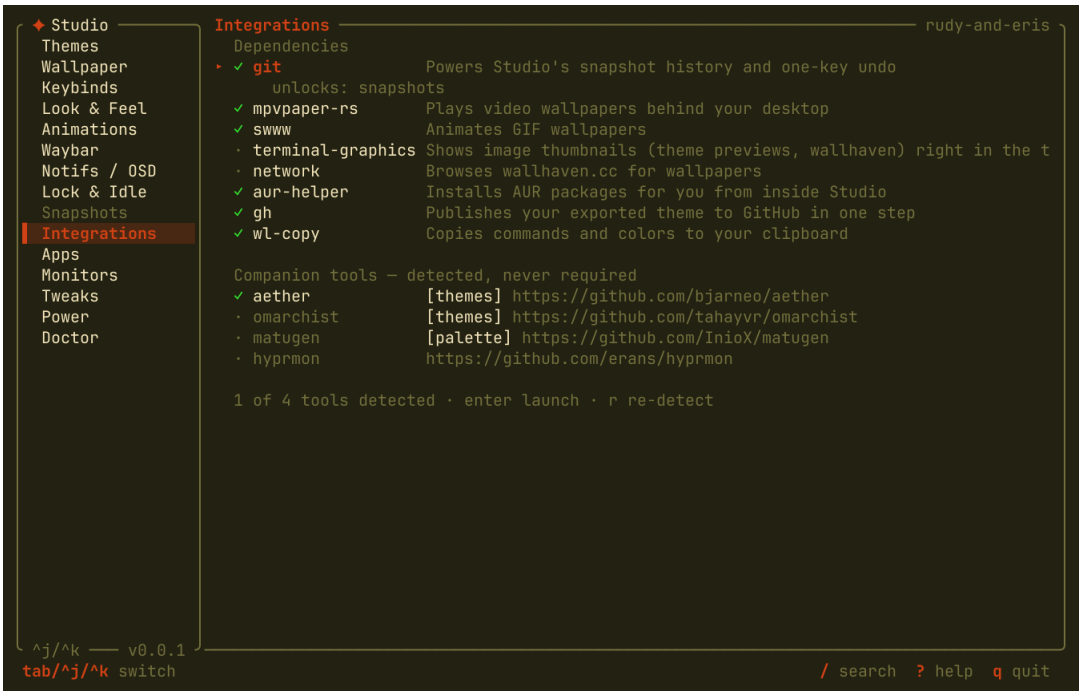
Safe removal of bundled apps and web apps: a `pacman -Rs` cascade preview, service handling and a restore log, with no bulk-confirm bypass.

10. Quick Tweaks



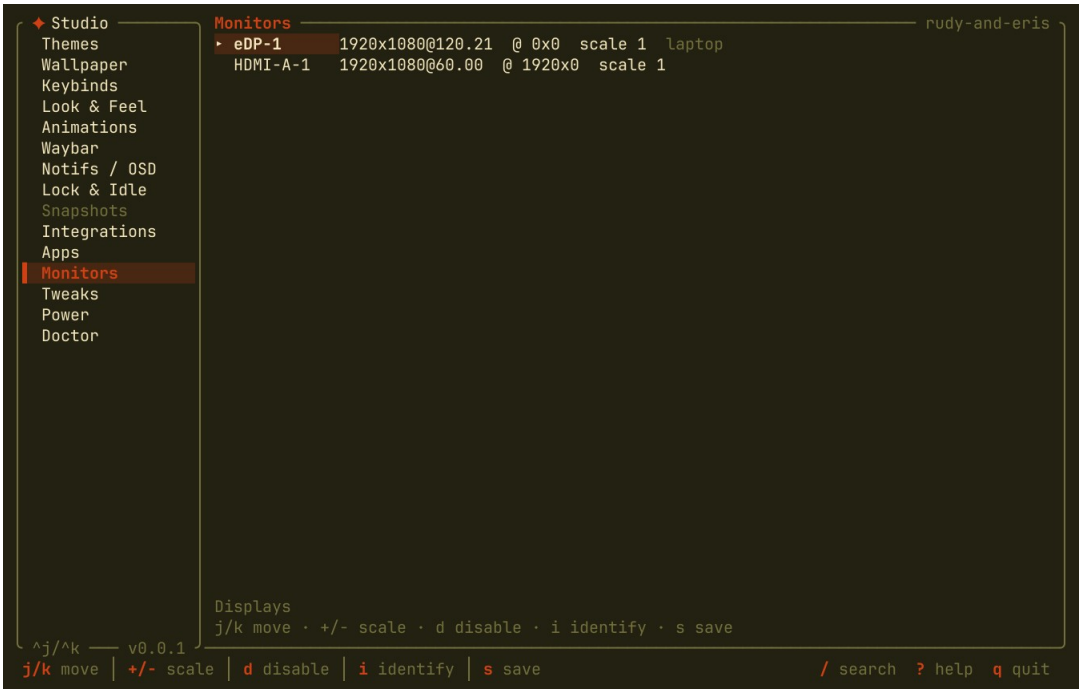
One-key reversible toggles; each tweak owns a uniquely named managed block and reports its live state.

11. Integrations Screen



Dependency health with the exact install command for anything missing, plus detection of companion tools such as Aether, Omarchist, matugen and hyprmon.

12. Doctor



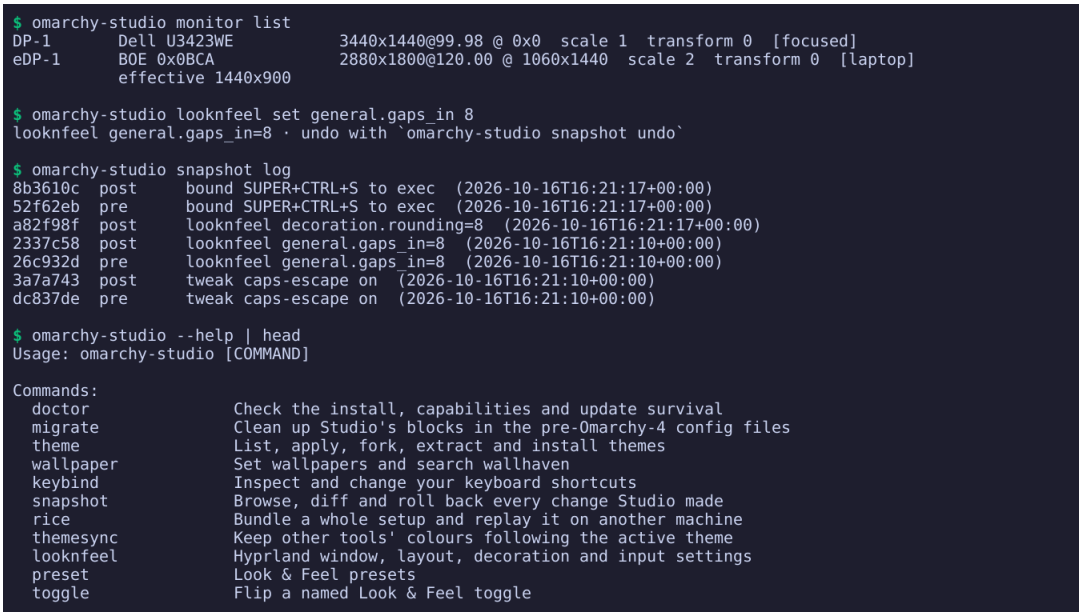
One health view: system facts, capability probes, hook status and a drift report showing whether tracked files still match the last snapshot.

13. First-run On-ramp



A fresh install opens on a welcome card with three optional, reversible set-up steps.

14. CLI Twin



The same engine from the command line: monitor listing, a look-and-feel change that prints its undo hint, the snapshot log and the command list.

7. Comparative Analysis

1. FEATURE COVERAGE MATRIX

Table 4: Coverage of customisation areas (from each project's documentation)

Capability	Omarchy Studio	A La Carchy	Aether	Omarchist	hyprmon	Omarchy menu
Theme browse & apply	Yes	Yes	Partial	Yes	No	Yes
Theme from wallpaper	Yes	Yes	Yes	No	No	No
Keybind editing	Yes	Yes	No	No	No	Partial
Hyprland look & feel	Yes	Yes	No	Partial	No	Partial
Animations	Yes	No	No	No	No	Partial
Status bar layout	Yes	Partial	No	Partial	No	No
Notifications / OSD / idle	Yes	No	No	No	No	No
Monitors	Yes	Yes	No	No	Yes	Partial
App removal	Yes	Yes	No	No	No	Partial
ASUS ROG hardware	No	Yes	No	No	No	No

“Partial” for the Omarchy menu means it opens the relevant file in a text editor; for A La Carchy's bar it means a few on/off toggles rather than a layout editor. A La Carchy is honestly the better tool for ASUS ROG laptops today.

2. SAFETY & ENGINEERING COMPARISON

Table 5: Safety and engineering comparison

Aspect	Manual editing	A La Carchy	Omarchy Studio
Form	Text editor + docs	One 10.8k-line bash script	Rust engine crate + TUI/CLI binary
Preview	None	None	Live (hyprctl keyword, image previews)
Undo	None (manual .bak)	tar.gz backup of 8 dirs	git timeline, diff, any-point restore
Apply safety	None	Confirmation prompts	Verify → automatic rollback, bar watchdog
Hand-edits	—	Edited in place	Preserved byte-for-byte
Survives omarchy-update	Only user files	Mostly (edits some vendored files)	Always
Omarchy 4 support	—	Waiting for v4	Lua dialect + shell adapter
Automated tests	—	None	477 tests + e2e in CI

3. QUALITATIVE ANALYSIS (STRENGTHS vs. WEAKNESSES)

Tool	Key Strengths	Key Weaknesses
Omarchy Studio	Covers visual and behavioural customisation in one place; live preview; verified apply with rollback; git-backed undo; CLI parity; Omarchy 3 and 4.	Terminal-only (no GUI yet); Omarchy-specific; alpha status; no ROG hardware controls.
A La Carchy	Very broad feature set incl. ASUS ROG; zero install (curl | bash).	No verified rollback; edits some vendored files; no tests; paused for Omarchy 4.
Aether	Excellent palette extraction, filters and presets in a polished GUI.	Colour theming only; heavier GUI stack.
Omarchist	Visual theme designer with native GPUI rendering.	Early development; no keybinds, animations or notifications.
hyprmon	Focused, capable monitor layout TUI.	Single purpose.
Manual editing	Unlimited flexibility; no extra tools.	Five dialects, no discoverability, no preview, no undo, easy to break.

8. Conclusion

The development of Omarchy Studio demonstrates that deep desktop customisation does not have to be risky or require fluency in a stack of configuration languages. By putting a schema-driven terminal interface and a matching CLI on top of a single verified apply pipeline, we turned the most error-prone activity on a Hyprland desktop — hand-editing configuration files — into a guided, previewable and fully reversible workflow.

The project's central design decision was to treat trust as the product. Lossless, comment-preserving parsers ensure that the user's own edits survive byte-for-byte; the never-fight-Omarchy rule ensures that system updates and Studio's changes cannot clobber each other; verification after every reload with automatic rollback ensures that a bad change cannot strand the user; and a git-backed snapshot store makes every change in the system's history inspectable and undoable. These guarantees are backed by 477 automated tests and end-to-end terminal journeys that run on every commit.

Studio also proved adaptable: when Omarchy 4 replaced the Hyprland configuration language, the menu system and four desktop components, the layered architecture allowed the change to be absorbed in the adapter and config engine while the modules, pipeline and snapshot store remained unchanged. The tool has been released publicly as open-source software, packaged for the AUR and used by members of the Omarchy community.

Overall, the project shows how established ideas from configuration-error research, interface design and version control can be combined into a practical tool that makes a modern Linux desktop approachable for newcomers while remaining fast and scriptable for experienced users.

9. Future Scope

While the current system covers the full customisation journey on Omarchy, several avenues remain for future enhancement. The most significant is a **graphical frontend**: because `studio-core` never prints, draws or prompts, a GUI (or a Walker-native launcher flow) can be built over the same engine without duplicating any domain logic, widening the audience to users who prefer the mouse.

On the platform side, a **generic-Hyprland mode** would decouple Studio from Omarchy's paths so that any Hyprland user could benefit from verified, undoable configuration. Deeper coverage of the Omarchy 4 Quickshell shell, per-module bar colours and AC/battery idle profiles are planned, together with an optional basic tier for **ASUS ROG** hardware (performance profiles, battery limits and keyboard brightness) behind capability gates.

The theming engine can be extended with the remaining palette-extraction modes (monochromatic, analogous, pastel, colourful, bright), single-theme export and sharing, and **theme sync** for more terminal tools beyond fzf and lazygit. Finally, the v1.0 milestone includes a written quick-start and tutorial, a “which file did that touch?” guide and a documented manual test protocol run on real hardware before every release — turning Studio from a working alpha into a tool a stranger can install from the AUR and use from start to finish.

10. References

[1] T. Xu and Y. Zhou, “Systems Approaches to Tackling Configuration Errors: A Survey,” ACM Computing Surveys, vol. 47, no. 4, Art. 70, 2015.

[2] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram and S. Pasupathy, “An Empirical Study on Configuration Errors in Commercial and Open Source Systems,” in Proc. 23rd ACM Symposium on Operating Systems Principles (SOSP), 2011, pp. 159–172.

[3] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy and R. Talwadker, “Hey, You Have Given Me Too Many Knobs!: Understanding and Dealing with Over-Designed Configuration in System Software,” in Proc. ESEC/FSE, 2015, pp. 307–319.

[4] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist and N. Diakopoulos, Designing the User Interface: Strategies for Effective Human-Computer Interaction, 6th ed. Pearson, 2016.

[5] P. Heckbert, “Color Image Quantization for Frame Buffer Display,” Computer Graphics (Proc. SIGGRAPH ’82), vol. 16, no. 3, pp. 297–307, 1982.

[6] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[7] Omarchy. [Online]. Available: https://omarchy.org. [Accessed: Oct. 2026].

[8] Hyprland Wiki. [Online]. Available: https://wiki.hypr.land. [Accessed: Oct. 2026].

[9] Ratatui — Rust library for terminal user interfaces. [Online]. Available: https://ratatui.rs. [Accessed: Oct. 2026].

[10] S. Klabnik and C. Nichols, The Rust Programming Language. [Online]. Available: https://doc.rust-lang.org/book/. [Accessed: Oct. 2026].

[11] toml_edit — format-preserving TOML parser. [Online]. Available: https://docs.rs/toml_edit. [Accessed: Oct. 2026].

[12] S. Chacon and B. Straub, Pro Git, 2nd ed. Apress. [Online]. Available: https://git-scm.com/book. [Accessed: Oct. 2026].

[13] wallhaven API v1 documentation. [Online]. Available: https://wallhaven.cc/help/api. [Accessed: Oct. 2026].

[14] D. Coffey, “A La Carchy.” [Online]. Available: https://github.com/DanielCoffey1/a-la-carchy. [Accessed: Oct. 2026].

[15] bjarneo, “Aether.” [Online]. Available: https://github.com/bjarneo/aether. [Accessed: Oct. 2026].

[16] Omarchy Studio source repository. [Online]. Available: https://github.com/arino08/omarchy-studio. [Accessed: Oct. 2026].

[17] tahayvr, “Omarchist.” [Online]. Available: https://github.com/tahayvr/omarchist. [Accessed: Oct. 2026].

[18] erans, “hyprmon.” [Online]. Available: https://github.com/erans/hyprmon. [Accessed: Oct. 2026].